

Image classification from scratch

Author: Abhishek Kumar

Linkedin: www.linkedin.com/in/abhishek-kumar-b2981b25b

Date created: 2026/04/15

Description: Training an image classifier from scratch on the Kaggle Cats vs Dogs dataset.

Reference links -

1. https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator
2. https://www.tensorflow.org/api_docs/python/tf/keras/utils/image_dataset_from_directory
3. <https://stackoverflow.com/questions/56450969/how-to-calculate-output-sizes-after-a-convolution-layer-in-a-configuration-file>
4. https://www.tensorflow.org/api_docs/python/tf/keras/Model

Dataset Link

1. Credits to :- <https://www.kaggle.com/datasets/salader/dogs-vs-cats?select=train>

Setting up Data Directory

```
In [1]: !mkdir -p ~/.kaggle/  
!cp kaggle.json ~/.kaggle/
```

```
In [2]: !kaggle datasets download -d salader/dogs-vs-cats
```

```
Warning: Your Kaggle API key is readable by other users on this system! To fix this,  
you can run 'chmod 600 /root/.kaggle/kaggle.json'  
Dataset URL: https://www.kaggle.com/datasets/salader/dogs-vs-cats  
License(s): unknown  
Downloading dogs-vs-cats.zip to /content  
 99% 1.05G/1.06G [00:05<00:00, 268MB/s]  
100% 1.06G/1.06G [00:06<00:00, 179MB/s]
```

```
In [4]: # !unzip /content/dogs-vs-cats.zip -d /content/
```

Installing the required libraries

```
In [6]: # !pip3 install tensorflow numpy matplotlib keras
```

Importing Packages

```
In [7]: import matplotlib.pyplot as plt
from PIL import Image
import random
import os

import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator

import random
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import load_img
from keras.utils import plot_model
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
import seaborn as sns
from tensorflow.keras.preprocessing.image import img_to_array, load_img
import numpy as np
```

Reading the data

```
In [8]: list_cats = ['/content/dogs_vs_cats/train/cats/cat.0.jpg', '/content/dogs_vs_cats/t
```

```
In [10]: # os.listdir('/content/dogs_vs_cats/train/dogs')
```

```
In [11]: random.sample(os.listdir('/content/dogs_vs_cats/train/dogs'), 5)
```

```
Out[11]: ['dog.10474.jpg',
'dog.12438.jpg',
'dog.8965.jpg',
'dog.8056.jpg',
'dog.6876.jpg']
```

```
In [11]:
```

```
In [12]: os.path.join("/content/dogs_vs_cats/train/cats/", 'cat.0.jpg')
```

```
Out[12]: '/content/dogs_vs_cats/train/cats/cat.0.jpg'
```

```
In [13]: # Placeholder for the directory path
cat_directory_path = '/content/dogs_vs_cats/train/cats'
dog_directory_path = '/content/dogs_vs_cats/train/dogs'
```

Data Analysis

```
In [14]: cat_images = os.listdir(cat_directory_path)
dog_images = os.listdir(dog_directory_path)

classes_animals = ['Cats', 'Dogs']
count = [len(cat_images), len(dog_images)]

print(count)
```

[10000, 10000]

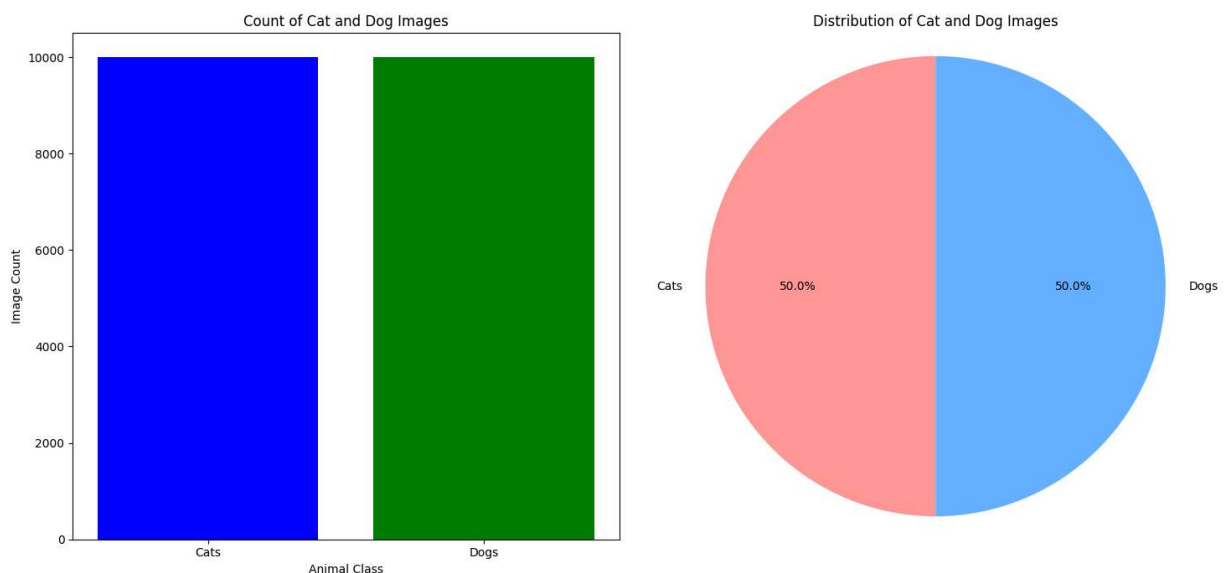
```
In [15]: # Creating both a bar chart and a pie chart side by side in a single figure

# Setting up a figure and axes for two subplots: one for the bar chart, one for the
fig, ax = plt.subplots(1, 2, figsize=(15, 7))

# Bar chart on the first subplot
ax[0].bar(classes_animals, count, color=['blue', 'green'])
ax[0].set_xlabel('Animal Class')
ax[0].set_ylabel('Image Count')
ax[0].set_title('Count of Cat and Dog Images')

# Pie chart on the second subplot
colors = ['#ff9999', '#66b3ff']
ax[1].pie(count, labels=classes_animals, colors=colors, autopct='%1.1f%%', startang
ax[1].set_title('Distribution of Cat and Dog Images')
ax[1].axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.

# Adjust the layout so that both subplots fit nicely
plt.tight_layout()
plt.show()
```



Loading Shapes and Channels

```
In [16]: import numpy as np
        from PIL import Image

        def image_to_array(image_path):
            """
            Read an image and convert it to a numpy array.

            Parameters:
            image_path (str): The path to the image file.

            Returns:
            np.array: The image as a numpy array.
            """
            try:
                with Image.open(image_path) as img:
                    image_array = np.array(img)
                    return image_array
            except IOError:
                print(f"Could not read the image file at {image_path}.")
                return None
```

```
In [17]: image_to_array('/content/dogs_vs_cats/train/dogs/dog.1000.jpg')
```

```
Out[17]: ndarray (375, 499, 3) show data
```



Using CV2

```
In [18]: image = '/content/dogs_vs_cats/train/dogs/dog.1000.jpg'

import cv2
```



```
img = cv2.imread(image)

# If the image is loaded successfully, print its pixel values
if img is not None:
    print(img)
    print(img.shape)
else:
    print("The image could not be loaded. Please check the path and file permission")
```

```

[[[ 4  8  9]
  [ 6 11 10]
  [11 13 13]
  ...
  [183 162 160]
  [184 161 159]
  [183 160 158]]

[[ 10 13 17]
 [ 11 15 16]
 [ 15 17 17]
  ...
 [180 159 157]
 [181 158 156]
 [180 157 155]]

[[ 8 12 17]
 [ 10 13 17]
 [ 14 16 17]
  ...
 [178 157 155]
 [179 156 154]
 [178 155 153]]

...

[[ 88 95 110]
 [ 89 96 111]
 [ 90 97 112]
  ...
 [101 111 128]
 [ 99 112 128]
 [ 99 112 128]]

[[ 88 95 110]
 [ 87 94 109]
 [ 88 95 110]
  ...
 [102 112 129]
 [100 113 129]
 [100 113 129]]

[[ 91 98 113]
 [ 88 95 110]
 [ 87 94 109]
  ...
 [ 98 108 125]
 [ 96 109 125]
 [ 96 109 125]]]
(375, 499, 3)

```

Using python

```

In [19]: # Import necessary Libraries
import numpy as np

```

```
from PIL import Image

# Load the image with PIL and convert to a NumPy array
img_pil = Image.open(image)
img_array = np.array(img_pil)

# Print the NumPy array of the image
print(img_array)

# Print the Shape of NumPy array of the image
print(img_array.shape)
```

```

[[[ 9  8  4]
  [ 10 11  6]
  [ 13 13 11]
  ...
  [160 162 183]
  [159 161 184]
  [158 160 183]]

[[ 17 13 10]
 [ 16 15 11]
 [ 17 17 15]
  ...
 [157 159 180]
 [156 158 181]
 [155 157 180]]

[[ 17 12  8]
 [ 17 13 10]
 [ 17 16 14]
  ...
 [155 157 178]
 [154 156 179]
 [153 155 178]]

...

[[110 95 88]
 [111 96 89]
 [112 97 90]
  ...
 [128 111 101]
 [128 112 99]
 [128 112 99]]

[[110 95 88]
 [109 94 87]
 [110 95 88]
  ...
 [129 112 102]
 [129 113 100]
 [129 113 100]]

[[113 98 91]
 [110 95 88]
 [109 94 87]
  ...
 [125 108 98]
 [125 109 96]
 [125 109 96]]]
(375, 499, 3)

```

```
In [1]: # for i,j,k in os.walk('/content/dogs_vs_cats'):
#       print(i,j,k)
```

```
In [22]: # !pip3 install opencv-python
```

Building Model

```
In [23]: # Set path to the dataset
base_dir = '/content/dogs_vs_cats/train'

# All images will be rescaled by 1./255
train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)
```

```
In [24]: # Using 20% of data for validation
train_generator = train_datagen.flow_from_directory(
    base_dir,
    target_size=(150, 150), # Resizing images to 150x150
    batch_size=20,
    class_mode='binary', # Since we use binary_crossentropy loss, we need binary labels
    subset='training') # Set as training data

validation_generator = train_datagen.flow_from_directory(
    base_dir,
    target_size=(150, 150),
    batch_size=20,
    class_mode='binary',
    subset='validation',
    shuffle=False # Disable shuffling for validation data
)
```

Found 16000 images belonging to 2 classes.

Found 4000 images belonging to 2 classes.

Getting Class Labels

```
In [25]: # Accessing class labels for the training data
train_class_labels = train_generator.class_indices
print("Training class labels:", train_class_labels)

# Accessing class labels for the validation data
validation_class_labels = validation_generator.class_indices
print("Validation class labels:", validation_class_labels)
```

Training class labels: {'cats': 0, 'dogs': 1}

Validation class labels: {'cats': 0, 'dogs': 1}

visualising the Batch Images

```
In [28]: images, labels = next(train_generator)
# images, labels
```

```
In [27]: import matplotlib.pyplot as plt

# Function to plot images and their labels
def plot_images_from_generator(generator, num_images=4):
```

```

# Get a batch of images and labels from the generator
images, labels = next(generator)

# Set up the plot
fig, axes = plt.subplots(1, num_images, figsize=(20,20))
axes = axes.flatten()

for img, label, ax in zip(images, labels, axes):
    # Image preprocessing might differ, adjust accordingly
    ax.imshow(img)
    # Set the title to the class label
    # Inverse the class indices dictionary to get class names from labels
    class_labels = dict((v,k) for k,v in generator.class_indices.items())
    ax.set_title(class_labels[int(label)])
    ax.axis('off')
plt.tight_layout()
plt.show()

# Now, call this function with your training generator
plot_images_from_generator(train_generator, num_images=4)

```



```
In [29]: train_generator.reset()
```

Basic CNN Model

```

In [30]: model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Sigmoid activation for binary classifi
])

model.summary()

```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:
107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 512)	3,211,776
dense_1 (Dense)	(None, 1)	513

Total params: 3,453,121 (13.17 MB)

Trainable params: 3,453,121 (13.17 MB)

Non-trainable params: 0 (0.00 B)

```
In [31]: from tensorflow.keras import models, layers

model = models.Sequential()

# Adding Layers one by one
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D(2, 2))

model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D(2, 2))

model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D(2, 2))

model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D(2, 2))

model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
```

```
model.add(layers.Dense(1, activation='sigmoid')) # Sigmoid activation for binary c
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_5 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_6 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_6 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_7 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dense_2 (Dense)	(None, 512)	3,211,776
dense_3 (Dense)	(None, 1)	513

Total params: 3,453,121 (13.17 MB)

Trainable params: 3,453,121 (13.17 MB)

Non-trainable params: 0 (0.00 B)

1. Multi-Class Classification Task:

If you have a multi-class classification task, the output layer should have as many units as there are classes, and the activation function should be softmax to ensure the output is a probability distribution over the classes. `model.add(layers.Dense(10, activation='softmax'))`

For multi-class classification, the output is a vector with the probability for each class, and you typically use a loss function like `categorical_crossentropy` or `sparse_categorical_crossentropy`.

2. Regression Task:

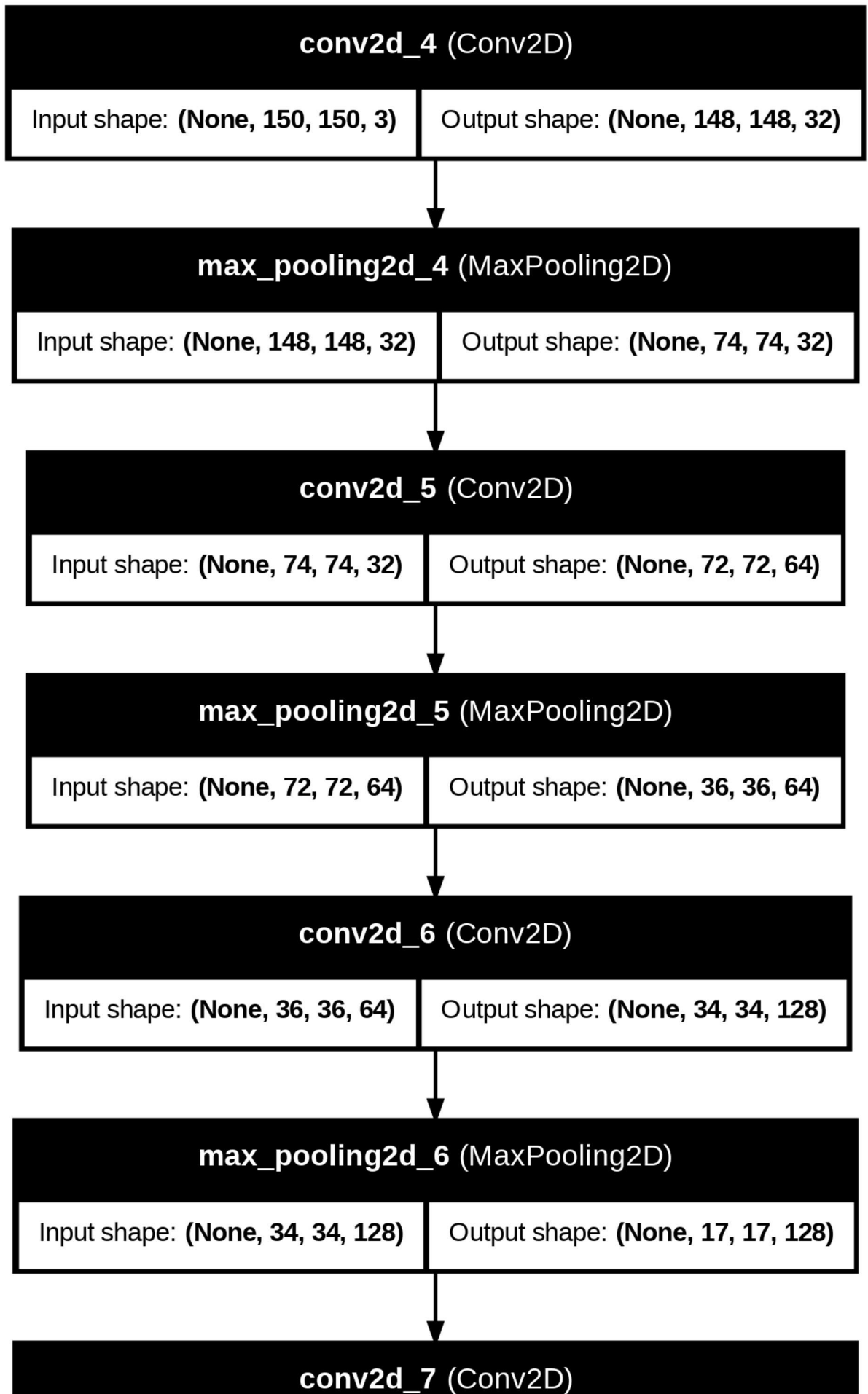
For a regression task, where the goal is to predict a continuous value, the output layer usually has a single unit with a linear activation function (or no activation function, which is equivalent to linear).

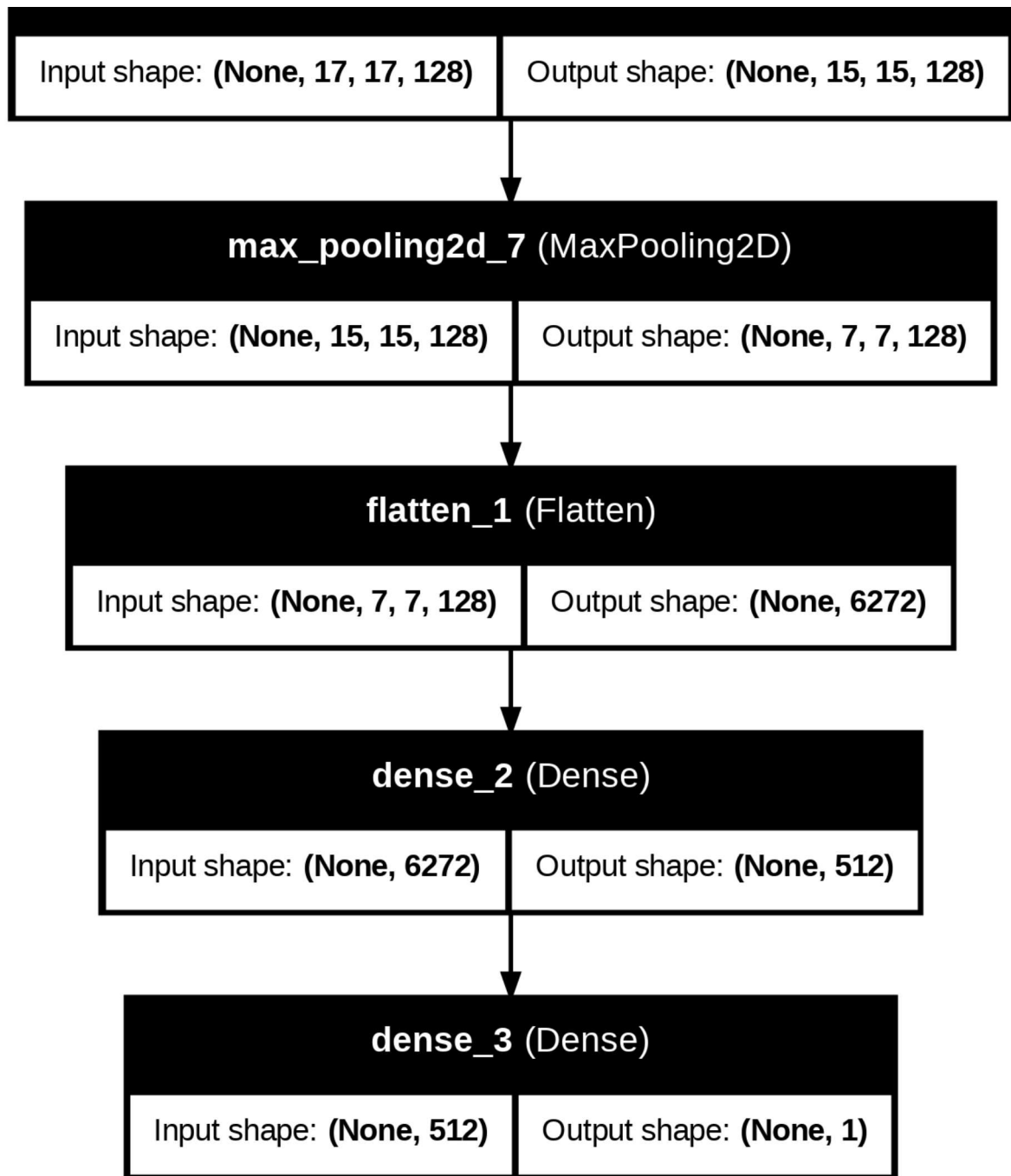
```
model.add(layers.Dense(1)) # Linear activation (default)
```


For regression, the output is a single continuous value, and you typically use a loss function like `mean_squared_error` or `mean_absolute_error`.

```
In [32]: plot_model(model, to_file = 'Architecture.png',  
                  show_shapes=True, show_layer_names=True)
```

Out[32]:





Parameter Calculation in CNN Layers

Conv2D Layer

The formula to calculate the number of parameters in a Conv2D layer is:

$$\text{Parameters} = (\text{kernel_height} \times \text{kernel_width} \times \text{input_channels} + 1) \times \text{number_of_filters}$$

The "+1" accounts for the bias term for each filter.

1. First Conv2D Layer

- Input shape: (150, 150, 3)
- Number of filters: 32
- Filter size: (3, 3)
- Parameters:

$$(3 \times 3 \times 3 + 1) \times 32 = 896$$

2. Second Conv2D Layer

- Input channels: 32 (from the previous Conv2D layer)
- Number of filters: 64
- Filter size: (3, 3)
- Parameters:

$$(3 \times 3 \times 32 + 1) \times 64 = 18,496$$

3. Third Conv2D Layer

- Input channels: 64
- Number of filters: 128
- Filter size: (3, 3)
- Parameters:

$$(3 \times 3 \times 64 + 1) \times 128 = 73,856$$

4. Fourth Conv2D Layer

- Input channels: 128
- Number of filters: 128
- Filter size: (3, 3)
- Parameters:

$$(3 \times 3 \times 128 + 1) \times 128 = 147,584$$

MaxPooling2D Layer

MaxPooling layers do not have parameters; they only reduce the size of the input they're applied to, based on their pool size and stride.

Flatten Layer

The Flatten layer itself doesn't have parameters. It simply reshapes the input but does not affect the total parameter count.

Dense Layer

The formula for a Dense (fully connected) layer is:

$$\text{Parameters} = (\text{input_size} + 1) \times \text{output_size}$$

1. First Dense Layer

- Assuming Flatten output size: X (you'll need to calculate this based on the output of the last pooling layer).
- Output size: 512
- Parameters:

$$(X + 1) \times 512$$

2. Second Dense Layer

- Input size: 512
- Output size: 1 (for binary classification)
- Parameters:

$$(512 + 1) \times 1 = 513$$

```
In [33]: model.compile(loss='binary_crossentropy',
                      optimizer='adam',
                      metrics=['accuracy'])
```

```
In [34]: train_generator.samples
```

```
Out[34]: 16000
```

```
In [35]: validation_generator = train_datagen.flow_from_directory(
        base_dir,
        target_size=(150, 150),
        batch_size=20,
        class_mode='binary',
        subset='validation',
        shuffle=False # Disable shuffling for validation data
    )
```

Found 4000 images belonging to 2 classes.

```
In [36]: # Set the number of epochs
epochs = 20

# You can use this but keras automatically calculates the steps per epoch so you do
'''
# Train the model
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size
)
```

```
'''  
history = model.fit(  
    train_generator,  
    epochs=epochs,  
    validation_data=validation_generator  
)
```

Epoch 1/20

```
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_  
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**  
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`  
, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore  
d.  
    self._warn_if_super_not_called()
```

800/800 ————— 39s 41ms/step - accuracy: 0.5366 - loss: 0.6921 - val_accuracy: 0.6330 - val_loss: 0.6457
Epoch 2/20

800/800 ————— 34s 42ms/step - accuracy: 0.6938 - loss: 0.5813 - val_accuracy: 0.7628 - val_loss: 0.4887
Epoch 3/20

800/800 ————— 32s 39ms/step - accuracy: 0.7787 - loss: 0.4609 - val_accuracy: 0.8163 - val_loss: 0.4108
Epoch 4/20

800/800 ————— 46s 46ms/step - accuracy: 0.8302 - loss: 0.3789 - val_accuracy: 0.8250 - val_loss: 0.3946
Epoch 5/20

800/800 ————— 31s 39ms/step - accuracy: 0.8527 - loss: 0.3273 - val_accuracy: 0.8558 - val_loss: 0.3495
Epoch 6/20

800/800 ————— 35s 44ms/step - accuracy: 0.8915 - loss: 0.2552 - val_accuracy: 0.8677 - val_loss: 0.3233
Epoch 7/20

800/800 ————— 31s 39ms/step - accuracy: 0.9136 - loss: 0.2058 - val_accuracy: 0.8378 - val_loss: 0.4171
Epoch 8/20

800/800 ————— 33s 41ms/step - accuracy: 0.9423 - loss: 0.1410 - val_accuracy: 0.8615 - val_loss: 0.4091
Epoch 9/20

800/800 ————— 31s 39ms/step - accuracy: 0.9634 - loss: 0.1014 - val_accuracy: 0.8543 - val_loss: 0.5318
Epoch 10/20

800/800 ————— 32s 40ms/step - accuracy: 0.9693 - loss: 0.0765 - val_accuracy: 0.8493 - val_loss: 0.5216
Epoch 11/20

800/800 ————— 31s 38ms/step - accuracy: 0.9832 - loss: 0.0500 - val_accuracy: 0.8605 - val_loss: 0.5422
Epoch 12/20

800/800 ————— 43s 41ms/step - accuracy: 0.9816 - loss: 0.0520 - val_accuracy: 0.8583 - val_loss: 0.6108
Epoch 13/20

800/800 ————— 36s 45ms/step - accuracy: 0.9876 - loss: 0.0395 - val_accuracy: 0.8670 - val_loss: 0.6858
Epoch 14/20

800/800 ————— 31s 39ms/step - accuracy: 0.9885 - loss: 0.0336 - val_accuracy: 0.8612 - val_loss: 0.6836
Epoch 15/20

800/800 ————— 31s 39ms/step - accuracy: 0.9843 - loss: 0.0493 - val_accuracy: 0.8655 - val_loss: 0.7378
Epoch 16/20

800/800 ————— 31s 39ms/step - accuracy: 0.9882 - loss: 0.0337 - val_accuracy: 0.8528 - val_loss: 0.7370
Epoch 17/20

800/800 ————— 32s 40ms/step - accuracy: 0.9893 - loss: 0.0323 - val_accuracy: 0.8602 - val_loss: 0.7916
Epoch 18/20

800/800 ————— 31s 39ms/step - accuracy: 0.9884 - loss: 0.0319 - val_accuracy: 0.8570 - val_loss: 0.7844
Epoch 19/20

800/800 ————— 41s 39ms/step - accuracy: 0.9944 - loss: 0.0180 - val_accuracy: 0.8635 - val_loss: 0.7384

Epoch 20/20

800/800  **30s** 38ms/step - accuracy: 0.9918 - loss: 0.0255 - val_accuracy: 0.8635 - val_loss: 0.8722

In [37]: `history`

Out[37]: `<keras.src.callbacks.history.History at 0x7ff150bd9f90>`

In [38]: `history.history`


```
Out[38]: {'accuracy': [0.5831249952316284,
0.7172499895095825,
0.7880625128746033,
0.8316875100135803,
0.8584374785423279,
0.886062502861023,
0.9141250252723694,
0.9363124966621399,
0.9574999809265137,
0.9682499766349792,
0.9793750047683716,
0.9790624976158142,
0.9865624904632568,
0.9861249923706055,
0.9865624904632568,
0.9833750128746033,
0.9895625114440918,
0.9883124828338623,
0.9899374842643738,
0.9911875128746033],
'loss': [0.6634442806243896,
0.5554206967353821,
0.4530552625656128,
0.37788939476013184,
0.319029301404953,
0.2651565372943878,
0.20846447348594666,
0.15233305096626282,
0.11048063635826111,
0.08065012842416763,
0.05903666093945503,
0.061234250664711,
0.04477083310484886,
0.03881116956472397,
0.03920281305909157,
0.04630615562200546,
0.029857613146305084,
0.03218287229537964,
0.030147626996040344,
0.02810412459075451],
'val_accuracy': [0.6330000162124634,
0.7627500295639038,
0.8162500262260437,
0.824999988079071,
0.8557500243186951,
0.8677499890327454,
0.8377500176429749,
0.8615000247955322,
0.8542500138282776,
0.8492500185966492,
0.8604999780654907,
0.8582500219345093,
0.8669999837875366,
0.8612499833106995,
0.8654999732971191,
0.8527500033378601,
```

```

0.8602499961853027,
0.8569999933242798,
0.8634999990463257,
0.8634999990463257],
'val_loss': [0.6456539034843445,
0.488707035779953,
0.41075101494789124,
0.3945632576942444,
0.3495444655418396,
0.32334524393081665,
0.4171072244644165,
0.40910983085632324,
0.5318498015403748,
0.5216168165206909,
0.542248010635376,
0.6108006834983826,
0.685768723487854,
0.6836449503898621,
0.7378435134887695,
0.7369614839553833,
0.7915722131729126,
0.7844181656837463,
0.7384135723114014,
0.8721953630447388]}

```

```

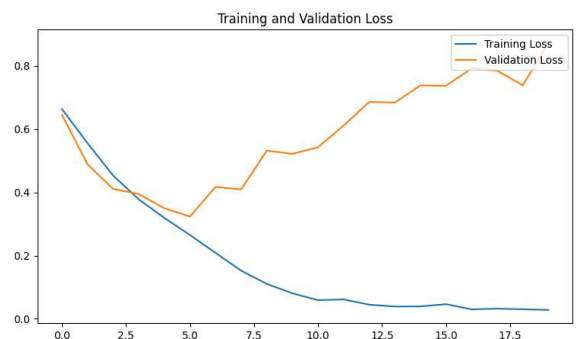
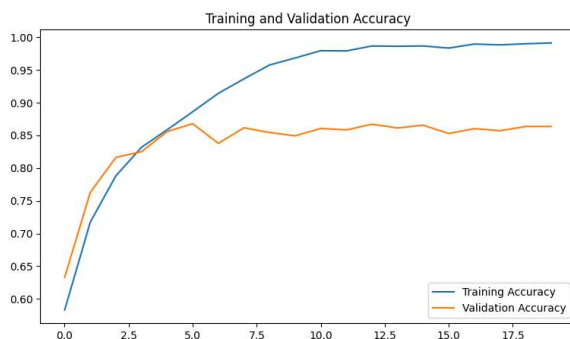
In [39]: acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(len(acc))

plt.figure(figsize=(20, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()

```



Model Evaluation

Initiating Test Generator

```
In [40]: # Define the test data generator
test_datagen = ImageDataGenerator(rescale=1./255) # Only rescaling for test data

test_dir = '/content/dogs_vs_cats/test' # Path to your test directory

# Create the test generator
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(150, 150), # Should match the input size of your network
    batch_size=20, # Can adjust based on your memory constraints
    class_mode='binary', # We do not need the labels here
    shuffle=False) # Do not shuffle to maintain the order of files
```

Found 5000 images belonging to 2 classes.

```
In [41]: # Evaluating the model
test_loss, test_accuracy = model.evaluate(test_generator, steps=len(test_generator))

print(f'Test Loss: {test_loss}')
print(f'Test Accuracy: {test_accuracy}')
```

250/250 ————— 7s 30ms/step - accuracy: 0.8860 - loss: 0.6264
Test Loss: 0.8278293013572693
Test Accuracy: 0.8682000041007996

Making test predictions

```
In [42]: np.ceil(test_generator.samples/test_generator.batch_size)
```

Out[42]: np.float64(250.0)

```
In [43]: probabilities = model.predict(test_generator)
probabilities
```

250/250 ————— 8s 30ms/step

```
Out[43]: array([[6.1781945e-22],
                [1.1525835e-15],
                [3.9399712e-12],
                ...,
                [1.0000000e+00],
                [5.0114739e-01],
                [9.7393936e-01]], dtype=float32)
```

```
In [46]: probabilities = model.predict(test_generator,
                                         steps=int(np.ceil(test_generator.samples/test_generat
probabilities
```

250/250 ————— 7s 28ms/step

```
Out[46]: array([[6.1781945e-22],
                [1.1525835e-15],
                [3.9399712e-12],
                ...,
                [1.0000000e+00],
                [5.0114739e-01],
                [9.7393936e-01]], dtype=float32)
```

```
In [48]: # Predict probabilities for the positive class
probabilities = model.predict(test_generator,
                              steps=int(test_generator.samples / test_generator.batch_size))

# The output is the probability of the positive class (assuming binary classification)
# so we will threshold it at 0.5 to decide the predicted class
predicted_classes = (probabilities > 0.5).astype(int).squeeze()

# To match with the filenames or other identifiers, you can extract the file names
filenames = test_generator.filenames

# If you need the actual class labels instead of 0/1, you can map the predicted classes to labels
class_labels = list(test_generator.class_indices.keys())
predicted_labels = [class_labels[i] for i in predicted_classes]

# Now you can inspect the results:
# for i in range(len(filenames)):
#     print(f"File: {filenames[i]} - Predicted: {predicted_labels[i]}")
```

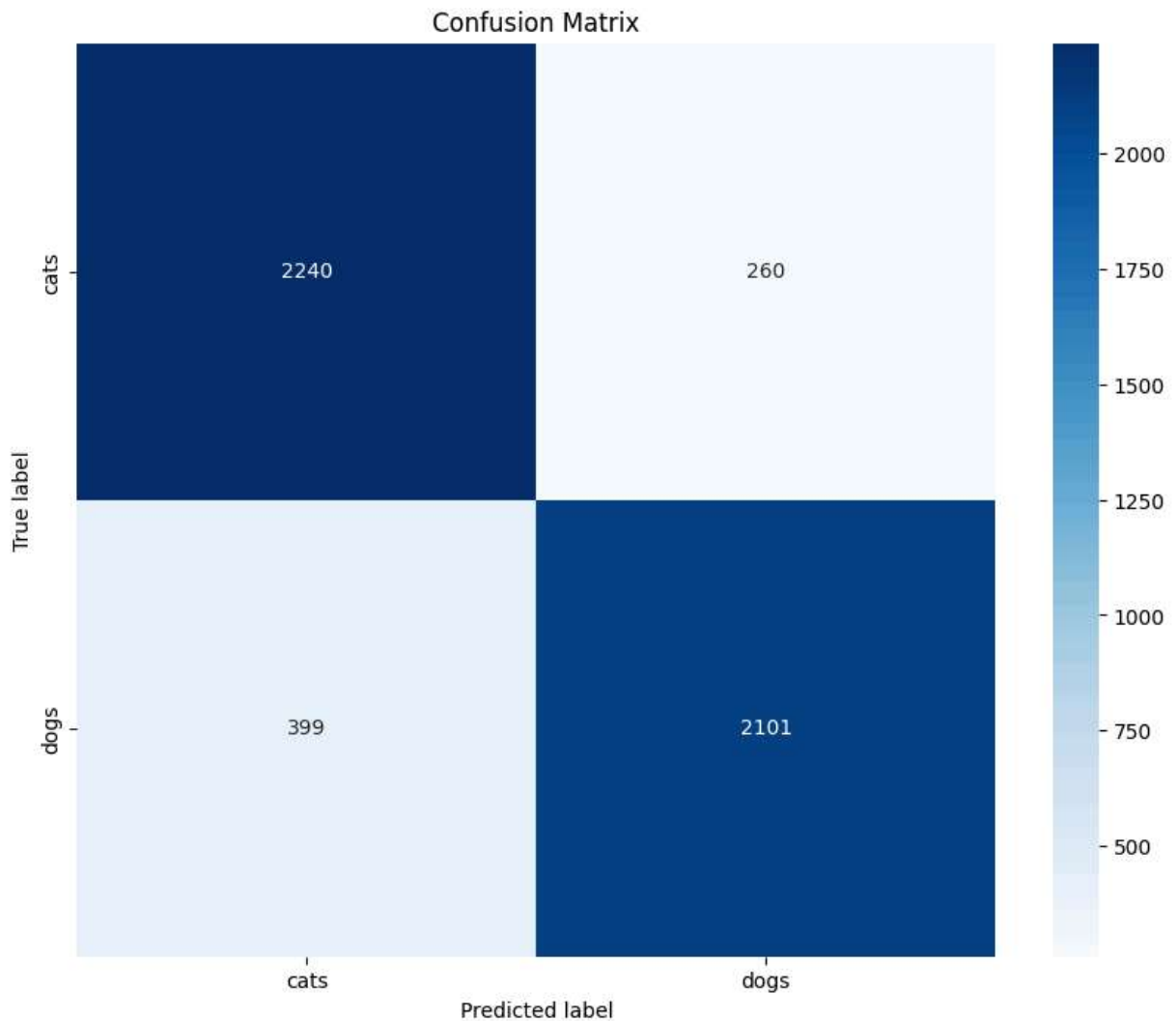
250/250 ————— 8s 31ms/step

Plotting confusion Matrix

```
In [49]: # Get the true labels from the test generator
true_classes = test_generator.classes

# Generate the confusion matrix
cm = confusion_matrix(true_classes, predicted_classes)

# Plotting with seaborn
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_labels, yticklabels=class_labels)
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.show()
```



Inference

```
In [50]: # Evaluate the model on the test data
loss, accuracy = model.evaluate(test_generator)

# Print the results
print(f"Test accuracy: {accuracy*100:.2f}%")
print(f"Test loss: {loss:.2f}")
```

250/250 ————— 11s 45ms/step - accuracy: 0.8860 - loss: 0.6264
Test accuracy: 86.82%
Test loss: 0.83

Single Image Prediction

```
In [51]: def make_prediction(image_path):
img = load_img(image_path, target_size=(150, 150)) # Resizing the image
img_array = img_to_array(img) # Converting to array
img_array = np.expand_dims(img_array, axis=0) # Add batch dimension
img_array /= 255. # Rescale the image similarly as we did for the training dat
```

```

prediction = model.predict(img_array)
print("Prediction:", prediction)

# Original image
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title('Original Image')
plt.axis('off')

# Prediction
plt.subplot(1, 2, 2)
if prediction[0][0] > 0.5:
    plt.title('Predicted: Dog')
else:
    plt.title('Predicted: Cat')
plt.imshow(img) # Show the same image as prediction
plt.axis('off')

plt.show()

```

```

In [53]: # Load and preprocess the image
img_path = '/content/kitty-cat-kitten-pet-45201.jpeg'
make_prediction(img_path)

```

1/1 ————— 0s 29ms/step

Prediction: [[5.351961e-31]]

Original Image



Predicted: Cat



```

In [52]: # Load and preprocess the image
img_path = '/content/dogs_vs_cats/test/dogs/dog.100.jpg'
make_prediction(img_path)

```

1/1 ————— 1s 809ms/step

Prediction: [[0.99995804]]

Original Image



Predicted: Dog



MisClassifcaitons

```
In [54]: # Reset the test_generator before making predictions
test_generator.reset()

# Predict the whole test set
predictions = model.predict(test_generator, verbose=1)

# The predictions will be in the format of probabilities. You'll need to convert th
predicted_classes = (predictions > 0.5).astype(int).reshape(-1)
true_classes = test_generator.classes
misclassified_indices = np.where(predicted_classes != true_classes)[0]
misclassified_count = len(misclassified_indices)
print(f"Total misclassifications in the test set: {misclassified_count} out of {len
```

250/250 ————— 7s 27ms/step

Total misclassifications in the test set: 659 out of 5000

```
In [55]: # Generate predictions for all test images
predictions = model.predict(test_generator)
# Convert predictions to binary (0 or 1) based on a 0.5 threshold
predicted_classes = predictions > 0.5

# Retrieve the true labels from the generator
true_classes = test_generator.classes

# Filenames can also be useful for debugging
filenames = test_generator.filenames
# Identifying indices of misclassified images
misclassified_indices = np.where(predicted_classes.flatten() != true_classes)[0]

print(f"Total misclassifications: {len(misclassified_indices)}")

# Select a random subset of misclassified images
num_samples_to_display = 4
random_indices = random.sample(list(misclassified_indices), num_samples_to_display)
```

```

# Plot
plt.figure(figsize=(10, 10))
for i, index in enumerate(random_indices, 1):
    # Load image
    img_path = test_dir + '/' + filenames[index]
    img = load_img(img_path, target_size=(150, 150))

    # Predicted class
    predicted_class = "Dog" if predicted_classes[index] else "Cat"

    # Display
    plt.subplot(2, 2, i)
    plt.imshow(img)
    plt.title(f"Predicted: {predicted_class}")
    plt.axis('off')
plt.show()

```

250/250 ————— 7s 29ms/step

Total misclassifications: 659

Predicted: Cat



Predicted: Cat



Predicted: Cat



Predicted: Dog



Visualising the kernel feature maps.

```
In [56]: # Get all the layers in which conv is present.

for i in range(len(model.layers)):
    # check for convolutional layer
    if 'conv' not in model.layers[i].name:
        continue
    # get filter weights
    filters, biases = model.layers[i].get_weights()
    print("layer number",i,model.layers[i].name, filters.shape)

layer number 0 conv2d_4 (3, 3, 3, 32)
layer number 2 conv2d_5 (3, 3, 32, 64)
layer number 4 conv2d_6 (3, 3, 64, 128)
layer number 6 conv2d_7 (3, 3, 128, 128)
```

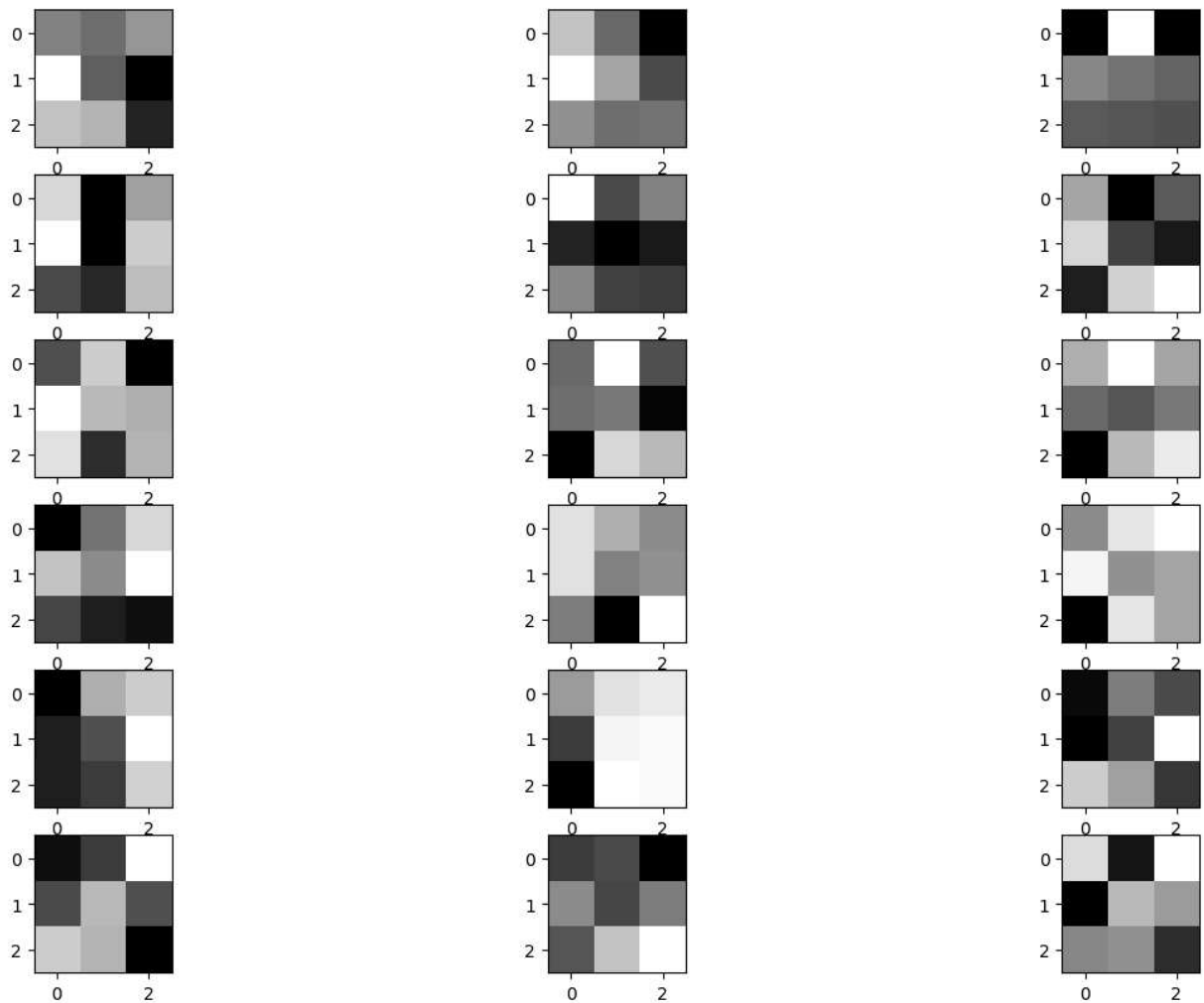
```
In [57]: # retrieve weights from the second hidden layer  
filters , bias = model.layers[2].get_weights()
```

```
In [58]: filters.shape, bias.shape
```

```
Out[58]: ((3, 3, 32, 64), (64,))
```

```
In [59]: # normalize filter values to 0-1 so we can visualize them  
f_min, f_max = filters.min(), filters.max()  
filters = (filters - f_min) / (f_max - f_min)
```

```
In [60]: n_filters=6  
ix=1  
fig = plt.figure(figsize=(15,10))  
for i in range(n_filters):  
    # get the filters  
    f = filters[:, :, :, i]  
    for j in range(3):  
        # subplot for 6 filters and 3 channels  
        plt.subplot(n_filters, 3, ix)  
        plt.imshow(f[:, :, j] , cmap='gray')  
        ix+=1  
#plot the filters  
plt.show()
```



```
In [61]: import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import tensorflow as tf

# Assuming 'model' is your pre-trained CNN model
model2 = tf.keras.Model(inputs=model.inputs, outputs=model.layers[1].output)

# Load and preprocess the image
image = load_img("/content/dogs_vs_cats/train/cats/cat.100.jpg", target_size=(150,
image = img_to_array(image)
image = np.expand_dims(image, axis=0)
image /= 255.0

# Predicting using model2 to get the features from the desired layer
features = model2.predict(image)

# Calculate the number of grids dynamically
num_feature_maps = features.shape[-1]
num_cols = 8 # You can adjust this number based on your preference
num_rows = num_feature_maps // num_cols + (1 if num_feature_maps % num_cols else 0)

fig = plt.figure(figsize=(20, 15))
for i in range(1, num_feature_maps + 1):
    ax = fig.add_subplot(num_rows, num_cols, i)
```

```
ax.imshow(features[0, :, :, i-1], cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

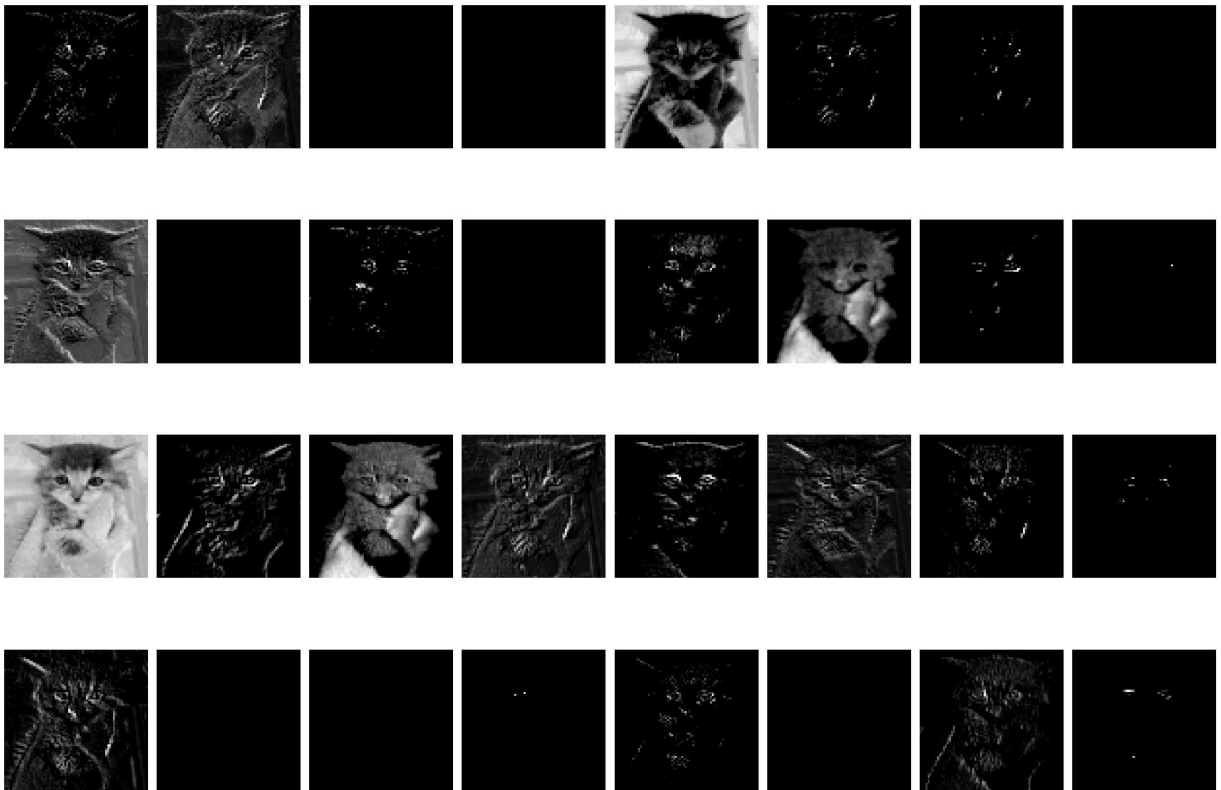
1/1 ————— 0s 157ms/step

/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of `inputs` doesn't match the expected structure.

Expected: ['keras_tensor_12']

Received: inputs=Tensor(shape=(1, 150, 150, 3))

warnings.warn(msg)



In [61]: